A Companion Deep-Dive for INT315 · Cluster Computing

THE HADOOP ECOSYSTEM

vs.

THE SPARK ECOSYSTEM

An In-Depth, Component-by-Component, Analogy-Driven Comparison

Architecture · Industry Standards · Conceptual Foundations

Plus: how INT312 (Big Data Fundamentals) connects to INT315 (Cluster Computing)

Session 2025-27

Companion Study Material — for course mates continuing from Big Data Fundamentals into Cluster Computing

Table of Contents

Where This Fits: From INT312 to INT315

If you took INT312: Big Data Fundamentals last semester, this document is written specifically as your bridge into INT315: Cluster Computing. The two courses share a subject — distributed data processing — but they look at it from two different altitudes, and understanding that difference up front will make everything below click faster.

- INT312 (Big Data Fundamentals) gave you BREADTH across the classic Hadoop world: what Big Data even is, HDFS storage, the MapReduce programming paradigm, YARN, and a tour of the storage layer options around it — Hive for SQL-style querying and HBase and Cassandra for NoSQL-style random access.
- INT315 (Cluster Computing) gives you DEPTH on the modern processing engine that grew up to replace hand-written MapReduce almost everywhere: Apache Spark — its Scala foundations, its RDD execution model, Spark SQL, Spark Streaming with Kafka, and Spark MLlib for machine learning at scale.

In other words: INT312 taught you the warehouse (storage systems) and the original assembly line (MapReduce). INT315 teaches you the faster, more flexible assembly line (Spark) that now runs in most of those same warehouses. This document exists to make that transition explicit, component by component, so nothing you learned last semester goes to waste — it becomes the foundation the new material stands on.

Same City, New Transit System

Picture INT312 as the semester where you learned the city of Big Data itself — its neighborhoods (HDFS storage, Hive's SQL district, HBase and Cassandra's NoSQL districts), and its original public bus system (MapReduce) that, while reliable, stops at every single corner and re-checks its paperwork at every stop.

INT315 is the semester a shiny new light-rail system (Spark) opens across that exact same city. The neighborhoods didn't move — HDFS is still HDFS, Hive still exists — but suddenly there's a much faster way to get across town, one that doesn't stop and re-file paperwork at every single corner, and can even handle types of trips (live streaming data, iterative machine learning) the old bus route was never built for.

This handbook is your transit map showing exactly which old bus routes map onto which new light-rail lines — and where a few genuinely new lines (like real-time streaming) didn't exist on the old map at all.

How to read this document if INT312 feels rusty

Every Hadoop-side section below (Module 2) is written as a self-contained refresher — you don't need your INT312 notes open to follow along. If a term feels totally new, it likely IS new (Spark introduced several concepts INT312 never covered), and that's called out explicitly wherever it happens.

MODULE 1 — What Do We Mean by an "Ecosystem"?

Before comparing Hadoop and Spark, it's worth being precise about what's actually being compared, because "Hadoop vs. Spark" is a slightly imprecise phrase that trips people up in interviews. Neither term refers to one single piece of software — each refers to a FAMILY of components that work together to solve the full range of Big Data problems: storing data, processing it in batch, querying it with SQL, handling random real-time access, and (increasingly) processing continuous streams and training machine learning models.

An "ecosystem" in this sense means: one or more STORAGE layers, one or more PROCESSING/COMPUTE engines, a RESOURCE MANAGEMENT layer that arbitrates who gets to use the cluster's hardware, and a ring of SUPPORTING TOOLS (query languages, workflow schedulers, coordination services) that make the whole thing usable by humans rather than just by low-level programmers.

- Storage layer — where the bytes physically live (HDFS, HBase, Cassandra, cloud object stores like S3).
- Processing / compute engine — what actually reads, transforms, and computes over that data (Hadoop MapReduce, Spark Core).
- Resource manager — what decides which application gets how much CPU/RAM on the shared cluster (YARN, Kubernetes, Spark Standalone).
- Supporting tools — what makes the engine usable day to day (Hive and Spark SQL for SQL-style access, Oozie/Airflow for scheduling, ZooKeeper for coordination).

The precise, interview-safe framing

The single most accurate one-line answer to "is Spark replacing Hadoop?" is: Spark primarily replaces the MAPREDUCE PROCESSING ENGINE layer, not the storage layer (HDFS) or the resource management layer (YARN) — both of which Spark commonly continues to use. This handbook is organized around exactly that layered structure so the comparison stays precise instead of collapsing into a vague "Spark is just better" answer, which under-informed candidates give and experienced interviewers immediately notice.

MODULE 2 — The Hadoop Ecosystem, Component by Component

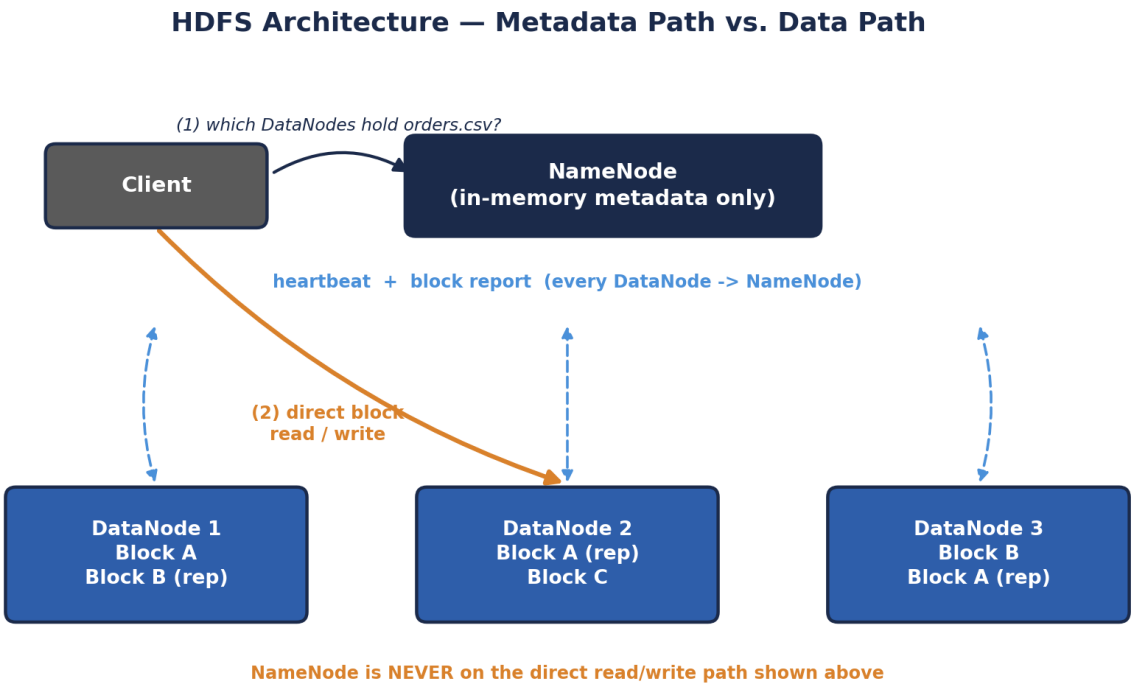
This module rebuilds the classic Hadoop stack from the ground up, layer by layer, with a focus on the ARCHITECTURE of each piece — the actual processes involved, what they talk to, and what fails if one of them goes down. If you took INT312, this will feel like a structured, deeper refresher rather than new material.

2.1 HDFS — Hadoop Distributed File System

HDFS is the storage foundation almost everything else in the Hadoop world is built on top of. Its design goal is simple to state and hard to build: store files far too large for any single machine, across many ordinary machines, while surviving individual machine failures without losing data or even pausing.

Architecture — the moving parts

- NameNode (master) — a single coordinating process holding the entire filesystem's METADATA in memory for speed: the directory tree, file names, permissions, and — critically — a mapping of which DataNodes hold which blocks of each file. It never stores actual file content.
- DataNode (worker) — one per storage machine; physically stores data blocks on local disk, sends periodic "heartbeat" signals to the NameNode to prove it's alive, and serves read/write requests for the blocks it holds.
- Secondary NameNode — a commonly misunderstood name; it is NOT a hot standby. It periodically merges the NameNode's edit logs into a checkpoint to keep NameNode restart times manageable. Modern Hadoop uses a proper Standby NameNode (via HDFS High Availability) for actual failover.
- Block — the unit of storage, 128MB by default in modern Hadoop; every file is split into blocks, and each block is independently replicated (3x by default) across different DataNodes, ideally on different racks, for fault tolerance.



HDFS Architecture — Metadata Path vs. Data Path

Walking through the diagram: step (1) is a lightweight METADATA-ONLY round trip — the client asks the NameNode a small question ("which DataNodes hold orders.csv?") and gets back a short list of locations. Step (2) is the actual heavy lifting, and it happens entirely between the client and the relevant DataNodes directly, completely bypassing the NameNode. This separation is precisely why the NameNode scales to coordinate petabytes of data despite being a single process: its workload is proportional to the number of metadata LOOKUPS, never to the volume of bytes actually flowing through the cluster. Each DataNode independently sends two kinds of signals upward — a frequent, lightweight heartbeat ("I'm still alive") and a periodic, heavier block report ("here is the complete, current list of every block I'm holding") — which together let the NameNode keep its in-memory metadata map accurate without ever touching the data itself.

```
# Core HDFS commands you'll use constantly
hdfs dfs -mkdir /user/student/data
hdfs dfs -put localfile.csv /user/student/data/
hdfs dfs -ls /user/student/data
hdfs dfs -cat /user/student/data/localfile.csv | head
hdfs dfs -get /user/student/data/localfile.csv ./downloaded.csv

# Check overall cluster health and see live/dead DataNodes
hdfs dfsadmin -report

# See exactly which blocks make up a file, and where each replica lives
hdfs fsck /user/student/data/localfile.csv -files -blocks -locations
```

The Central Library with Branch Warehouses

The NameNode is the library's card catalog desk — it knows the title, author, and exact shelf location (which DataNodes) of every book (file), but the desk itself holds no books. The DataNodes are the branch warehouses actually holding physical copies, each warehouse quietly phoning the front desk every few seconds just to say "still here, still fine" (the heartbeat).

If a warehouse stops calling in, the front desk quietly updates its records to route future requests to the OTHER warehouses holding a copy of that same book, and arranges for a replacement copy to be printed elsewhere to keep three copies in circulation. The front desk clerk is never seen personally carrying a book to a reader — once you ask which warehouse has it, you walk there yourself and pick it up directly.

Check yourself — 2.1

Q1. What specific information does the NameNode store, and what does it deliberately NOT store?

Q2. Why is the Secondary NameNode a misleading name, and what does it actually do?

Q3. If a DataNode stops sending heartbeats, what does the NameNode do about the blocks that DataNode was holding?

Hands-on Challenge:

Run `hdfs dfsadmin -report` on any Hadoop installation you have access to (or a sandbox/Docker image), and identify: total configured capacity, number of live DataNodes, and the current default replication factor shown in the output.

A closer look: what actually happens on a write

It's worth walking through an HDFS WRITE in a bit more detail, since it reveals a design decision students often gloss over: replication happens in a PIPELINE, not all at once from the client. When a client writes a new block, it does not send three separate copies to three separate DataNodes itself. Instead, the client sends the block to the FIRST DataNode in the pipeline, that DataNode forwards it to the second while simultaneously writing its own local copy, and the second forwards it to the third the same way. Only once all three DataNodes acknowledge the write does it count as successful. This pipelined approach spreads the network cost of replication across multiple machines instead of loading it entirely onto the client's own network connection, and is a detail that shows up surprisingly often in "how does HDFS actually replicate data" interview questions.

2.2 Hadoop MapReduce — Architecture & Terminology

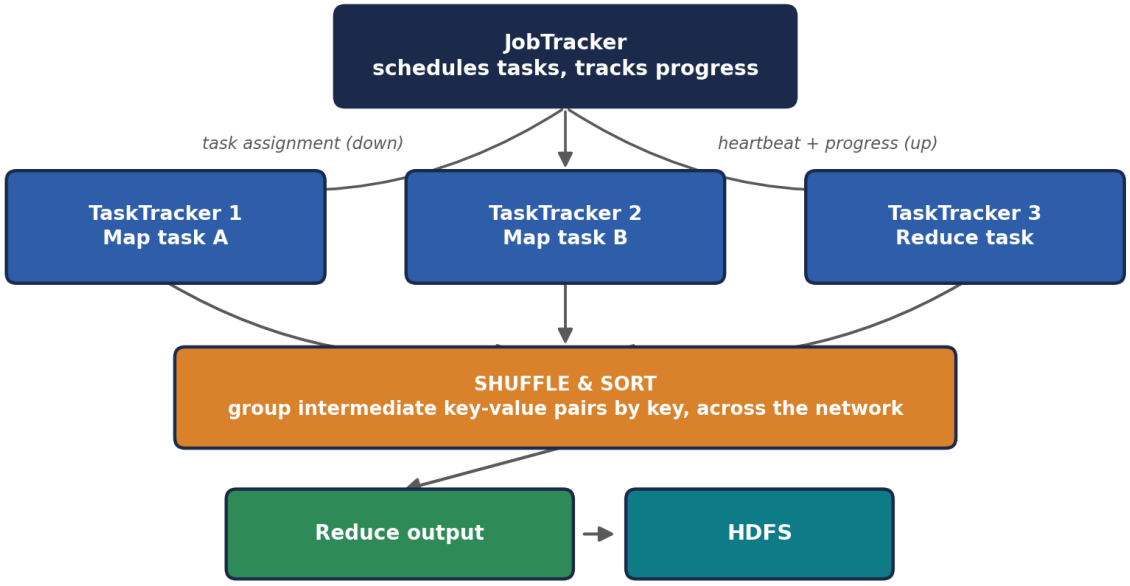
MapReduce is the original Hadoop processing engine — a programming MODEL (write a Map function and a Reduce function) backed by a specific execution architecture that schedules and runs those functions across the cluster.

Classic architecture (Hadoop 1.x) — JobTracker & TaskTracker

- JobTracker (master) — accepted submitted jobs, split them into tasks, and scheduled those tasks onto TaskTrackers across the cluster. It also handled resource tracking and monitoring, which — as you'll see in 2.3 — turned out to be too much responsibility for one process at scale.
- TaskTracker (worker) — ran the actual Map and Reduce tasks assigned to it, and reported progress and heartbeats back to the JobTracker.

> ⚠ **Why this architecture was retired**
>
> In Hadoop 1.x, the JobTracker was a single process simultaneously responsible for resource management AND job scheduling/monitoring for the ENTIRE cluster, across every running job. This became both a scalability ceiling (one process managing thousands of tasks cluster-wide) and a single point of failure for every running job at once. This exact pain point is precisely why YARN (Module 2.3) was introduced in Hadoop 2.x to split those responsibilities apart.

Classic MapReduce Job Flow — JobTracker / TaskTracker



Classic MapReduce Job Flow — JobTracker / TaskTracker

The diagram's shuffle arrow is doing a lot of work conceptually — it represents every intermediate (key, value) pair produced by every Map task potentially needing to travel across the network to whichever machine is running the Reduce task responsible for that key. This is the single most network- and disk-heavy phase of the entire job, and it's the exact phase Module 3.1 revisits when explaining why Spark's in-memory model sidesteps so much of this cost on multi-stage or iterative workloads.

Core MapReduce terminology

- InputSplit — a logical chunk of the input data (usually aligned to one HDFS block) assigned to one Map task.

- Mapper — the user-written class/function processing one InputSplit, emitting intermediate key-value pairs.
- Partitioner — decides which Reducer each intermediate key gets routed to during the shuffle (by default, a hash of the key).
- Combiner — an optional "mini-reducer" that runs locally on a Mapper's own output before the shuffle, reducing the volume of data that needs to cross the network (e.g., pre-summing word counts locally before shipping them).
- Reducer — the user-written class/function receiving all values for a given key and producing final output.
- Driver — the program that configures and submits the MapReduce job (input/output paths, Mapper/Reducer classes, etc.).

Word count on the command line (a INT312 favorite, revisited)

```
# Running the bundled example word-count job against a file already in HDFS
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
     wordcount /user/student/data/localfile.csv /user/student/output/wordcount

# View the result -- MapReduce writes output as part-r-00000, part-r-00001, etc.
hdfs dfs -cat /user/student/output/wordcount/part-r-00000 | head -20
```

Why the Combiner matters at scale

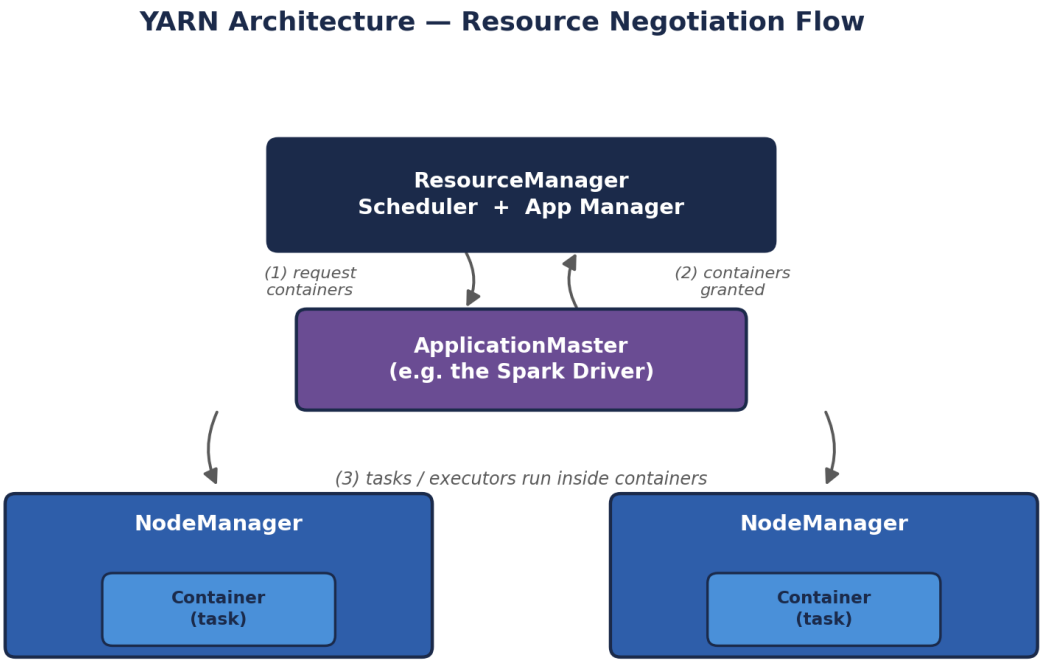
The Combiner is a small architectural detail with an outsized production impact: without it, EVERY single (word, 1) pair travels across the network during shuffle. With a Combiner doing local pre-aggregation on each Mapper's own output first, only the much smaller per-machine SUBTOTALS travel across the network. On genuinely large inputs, this can cut shuffle network traffic by orders of magnitude — a direct, practical example of Module 2.2's RAM/disk/network hierarchy from the Prerequisites handbook being exploited deliberately by the framework's design.

2.3 YARN — Yet Another Resource Negotiator

YARN was introduced specifically to split the JobTracker's overloaded responsibilities (2.2) into two cleanly separated roles: pure resource management, and per-application job monitoring. This split is exactly what allows YARN to host MULTIPLE different processing engines — classic MapReduce AND Spark AND others — on one shared cluster at the same time.

- ResourceManager (master, one per cluster) — tracks total available resources (CPU/RAM) across every NodeManager, and decides how much of that a given application is allowed to use. It does NOT track individual task-level progress within an application — that job is delegated.
- NodeManager (worker, one per machine) — manages resources and containers on its own machine, reporting available capacity to the ResourceManager and launching/monitoring containers on request.

- ApplicationMaster (one per running application) — a per-application coordinator that negotiates resources from the ResourceManager and tracks the progress of ITS OWN application's tasks specifically. When you run a Spark job on YARN, Spark's own driver plays this role.
- Container — the actual unit of allocated resource (a specific amount of CPU + RAM) on a specific NodeManager, in which a task or a Spark executor actually runs.



YARN Architecture — Resource Negotiation Flow

Two subtleties worth internalizing from this diagram. First, the ApplicationMaster itself runs INSIDE a container, just like any other task — YARN doesn't give it special treatment outside the resource-accounting system, which keeps the whole model consistent. Second, notice that the ResourceManager only ever talks to ApplicationMasters about COARSE resource grants ("you may have 4 more containers") — it has no idea what a Map task or a Spark stage actually IS. That fine-grained knowledge lives entirely inside each application's own ApplicationMaster, which is exactly the separation of concerns that fixed the Hadoop 1.x JobTracker's overload problem from Module 2.2.

> **The Convention Center, Revisited With a Proper Org Chart**
>
> *Back in the Prerequisites handbook, we compared a resource manager to a convention center's staffing coordinator. YARN is that coordinator with a proper org chart: the ResourceManager is the head coordinator who only cares about the big picture — "Kitchen A gets 40% of the staff today, Kitchen B gets 60%" — and never gets pulled into the weeds of any one kitchen's actual cooking schedule.*

Each kitchen (application) gets its OWN on-site manager — the ApplicationMaster — who negotiates with the head coordinator for staff, then handles that kitchen's actual minute-to-minute cooking schedule without bothering the head coordinator again unless more staff are needed. The NodeManagers are each individual room's floor supervisor, actually handing out aprons (containers) to whichever kitchen was approved to use that room.

🤔 Check yourself — 2.2 & 2.3

Q1. What two responsibilities did the Hadoop 1.x JobTracker combine, and why did that combination become a scalability problem?

Q2. In YARN, which component negotiates resources on behalf of one specific running application, and which component allocates resources across the whole cluster?

Q3. Explain, using the Combiner, one concrete way MapReduce's architecture deliberately reduces network shuffle traffic.

Hands-on Challenge:

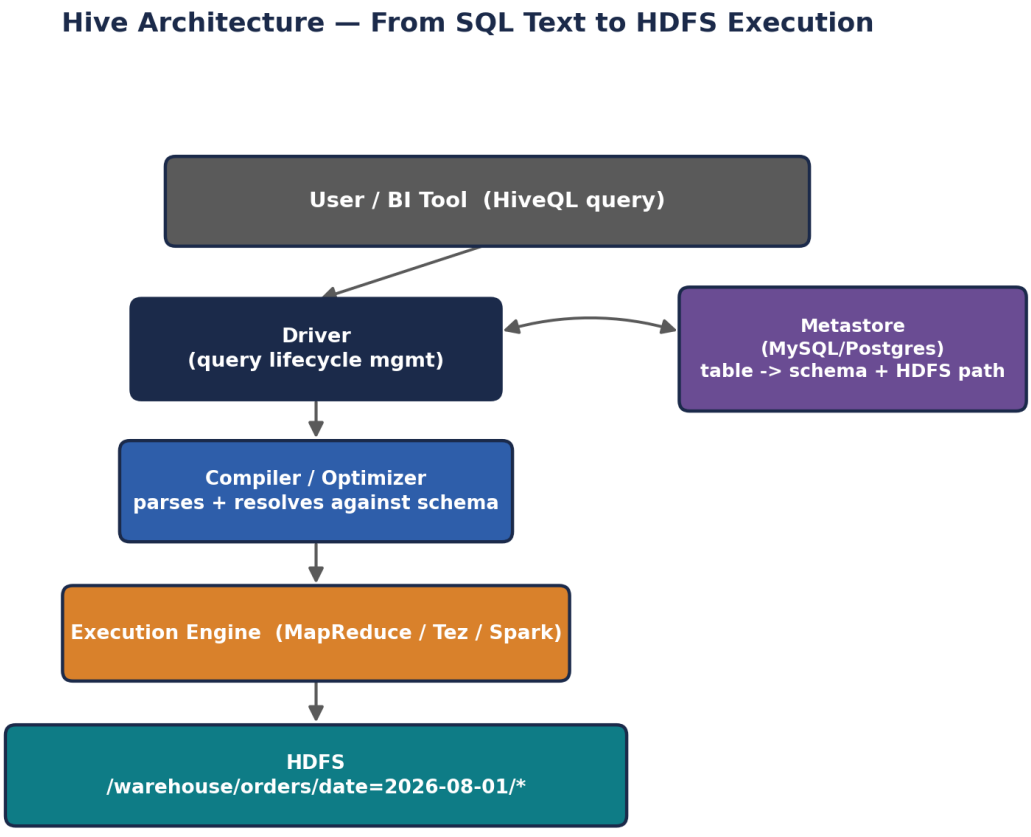
If you have a Hadoop installation available, submit the bundled word-count example against any text file, then open the YARN ResourceManager's web UI (typically http://<host>:8088) while or after it runs, and identify the ApplicationMaster entry for your job in the applications list.

2.4 Apache Hive — SQL on Top of Hadoop

Hive exists to solve a people problem as much as a technical one: most data analysts know SQL, not Java MapReduce code. Hive lets them write familiar SQL-like queries (HiveQL) against data sitting in HDFS, which Hive then translates into an execution plan — historically MapReduce jobs, though modern Hive installations can use Spark or a purpose-built engine called Tez instead.

Architecture

- Driver — receives the HiveQL query, manages its lifecycle (compilation, optimization, execution) end to end.
- Compiler — parses the query, checks it against the metastore's schema information, and builds an execution plan (traditionally a DAG of MapReduce jobs).
- Metastore — a separate relational database (commonly MySQL or PostgreSQL) storing Hive's SCHEMA metadata: table definitions, column types, and — critically — the mapping from Hive tables to their underlying file locations in HDFS. Hive tables are essentially a schema layer laid ON TOP OF plain files; the metastore is what makes that illusion of "tables" possible.
- Execution Engine — actually runs the compiled plan, historically as MapReduce jobs against HDFS.



Hive Architecture — From SQL Text to HDFS Execution

The Metastore box is worth staring at for a moment, because it's the piece that makes Hive's whole illusion work: without it, /warehouse/orders/ is just a directory of plain files with no inherent structure. The Metastore is the ONLY place that knows those files should be interpreted as a table called orders with a customer_id BIGINT column and an order_date partition key. This is also precisely why losing the Metastore is catastrophic for a Hive deployment even though the underlying data in HDFS remains completely intact — the schema mapping, not the data itself, is what would be lost.

Hive data organization: partitioning and bucketing

Two techniques you saw in INT312 are worth re-anchoring here because Spark SQL (Module 3.2) borrows both ideas directly:

- Partitioning — physically splitting a table's data into separate HDFS subdirectories based on a column's value (e.g., a directory per date: /warehouse/orders/date=2026-08-01/). A query filtering on that column can then skip reading irrelevant directories entirely, a technique called partition pruning.
- Bucketing — further splitting each partition (or the whole table) into a fixed number of files based on a hash of a column, which enables more efficient joins and sampling by guaranteeing rows with the same bucketed key land in predictably-numbered files.

```
-- HiveQL: creating a partitioned, bucketed table
CREATE TABLE orders (
  order_id BIGINT,
  customer_id BIGINT,
  amount DOUBLE
)
PARTITIONED BY (order_date STRING)
CLUSTERED BY (customer_id) INTO 8 BUCKETS
STORED AS ORC;

-- A query that benefits from partition pruning -- only reads the
-- order_date=2026-08-01 directory, not the whole table
SELECT customer_id, SUM(amount)
FROM orders
WHERE order_date = '2026-08-01'
GROUP BY customer_id;
```

> **The Filing Clerk Who Speaks Fluent SQL**
>
> *Imagine HDFS as a warehouse full of unlabeled boxes (raw files). Hive hires a filing clerk (the Metastore) whose entire job is remembering "boxes in aisle 7 rows 12-40 are actually the 'orders' table, and here's what's inside each column." You never touch the boxes directly -- you just ask the clerk a question in plain SQL, and the clerk (Driver + Compiler) translates that into precise instructions for the warehouse workers (the execution engine) about exactly which aisles to walk down.*

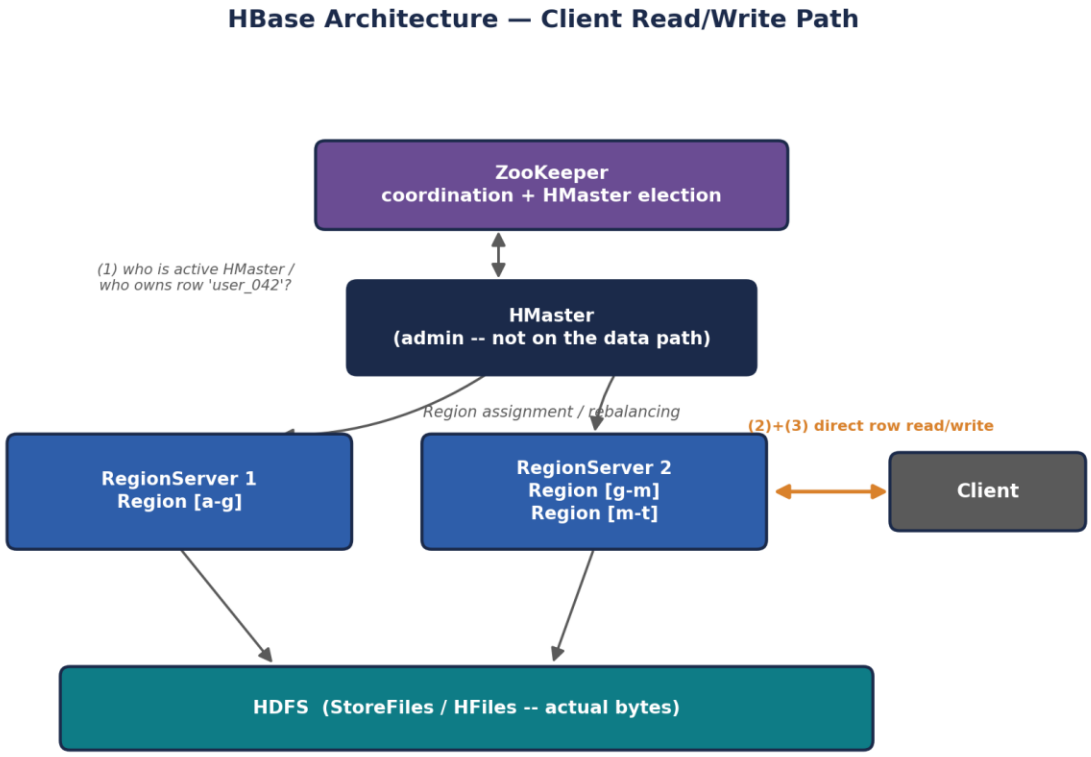
> *Partitioning is the clerk pre-sorting boxes into labeled AISLES by date, so a question about "just August 1st" means the workers walk straight to one aisle and ignore the rest of the warehouse entirely, rather than opening every box in the building.*

2.5 Apache HBase — Real-Time Random Access on Hadoop

HDFS and Hive are both built for large, sequential scans — reading big chunks of a file start to finish. Neither is designed to answer "give me this ONE specific row, right now, in milliseconds." HBase exists precisely to fill that gap: a distributed, column-oriented NoSQL database built on top of HDFS, modeled closely on Google's Bigtable design.

Architecture

- HMaster (master) — handles administrative operations: creating/deleting tables, and — critically — assigning Regions (see below) to specific RegionServers, rebalancing them as needed.
- RegionServer (worker) — serves read and write requests for the Regions assigned to it; this is where the actual data access happens, and where client requests are directed once ZooKeeper tells them where to look.
- Region — a horizontal slice of a table, holding a contiguous range of row keys; as a table grows, HBase automatically splits it into more Regions and redistributes them across RegionServers.
- ZooKeeper — a separate coordination service HBase depends on to track cluster state, elect the active HMaster (supporting failover), and help clients locate which RegionServer currently owns a given Region.



HBase Architecture — Client Read/Write Path

This diagram highlights a detail that mirrors HDFS's own NameNode/DataNode split from 2.1: the HMaster, like the NameNode, is deliberately kept OFF the hot read/write path. A client only consults ZooKeeper and (indirectly) the HMaster to learn WHICH RegionServer currently owns the row key it's after; every actual read or write after that lookup goes directly to that RegionServer. This is exactly why HBase can serve millisecond-latency requests at scale — the coordination layer is consulted rarely (and often cached by the client), while the data layer handles the actual traffic.

HBase's data model, briefly re-anchored

An HBase table is organized as: Row Key (uniquely identifies a row, and determines its sort order and which Region it lands in) -> Column Family (a grouping of related columns, defined at table-creation time and stored together on disk for efficiency) -> Column Qualifier (the specific column name within a family, which can be created dynamically per row, unlike a rigid relational schema) -> Cell (the actual stored value, versioned by timestamp).

```
# HBase shell -- general commands
create 'users', 'profile', 'activity'
put 'users', 'row1', 'profile:name', 'Aditi'
put 'users', 'row1', 'profile:age', '21'
get 'users', 'row1'
scan 'users'
```

```
# Filtering: prefix filter and single-column-value filter (as covered in INT312
Unit V)
scan 'users', {FILTER => "PrefixFilter('row1')"}
scan 'users', {FILTER =>
"SingleColumnValueFilter('profile','age',=,'binary:21')"}

# Setting a Time-To-Live on a column family -- cells auto-expire after N seconds
alter 'users', {NAME => 'activity', TTL => '86400'}
```

Why HBase and not just HDFS

A classic interview framing: HDFS is built for write-once, read-many, large SEQUENTIAL scans (think: a nightly batch job reading an entire day's log file). HBase is built for random, low-latency, individual-row READS AND WRITES at any time (think: looking up one specific user's profile the instant they load a webpage). Companies commonly run both side by side — HDFS/Hive for big analytical batch queries, HBase for the operational, user-facing lookups that need millisecond response times.

2.6 The Supporting Cast — Pig, Sqoop, Oozie, ZooKeeper

A few more tools round out the classic Hadoop ecosystem. You won't need deep architecture knowledge of these for INT315, but recognizing their exact role is important for both this comparison and for interviews.

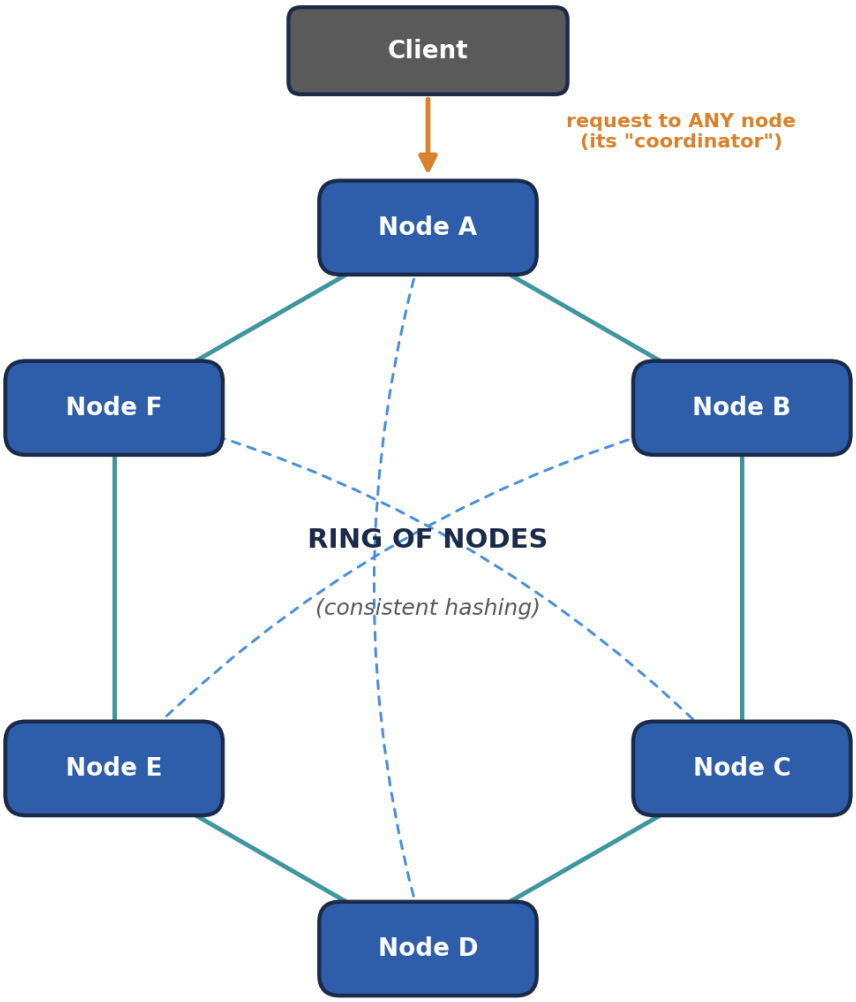
- Apache Pig — a scripting language (Pig Latin) offering a more procedural, dataflow-style alternative to SQL for expressing data transformation pipelines, which then compiles down to MapReduce (or Tez/Spark) jobs, similar in spirit to Hive but favored by engineers who think in terms of transformation STEPS rather than declarative SQL queries.
- Apache Sqoop — a bulk data-transfer tool specifically for moving data between HDFS and traditional relational databases (MySQL, Oracle, PostgreSQL) — importing existing enterprise data INTO the Hadoop world, and exporting processed results back OUT.
- Apache Oozie — a workflow scheduler for Hadoop jobs, letting you define multi-step pipelines with dependencies ("run the Hive aggregation only after the Sqoop import finishes, then trigger an HBase load"), often on a recurring schedule.
- Apache ZooKeeper — a lightweight, highly-available coordination service used BY other components (HBase's HMaster election, Kafka's older broker coordination, Hadoop's NameNode High Availability) to agree on shared cluster state, elect leaders, and manage distributed locks — it's infrastructure other tools lean on rather than something end users query directly.

2.7 A Note on Cassandra — Why It's Not Really "Hadoop Ecosystem"

If your INT312 course covered Apache Cassandra (Unit VI), it's worth being precise about where it actually sits, because it's commonly and incorrectly lumped in as "part of Hadoop."

Cassandra is architecturally quite different from everything else in this module: it has NO master node at all. Instead of a NameNode/HMaster-style coordinator, Cassandra uses a peer-to-peer, masterless architecture where every node is structurally equal, nodes discover each other and share cluster state via a gossip protocol, and data is distributed using consistent hashing across a ring of nodes rather than being coordinated by any single metadata service. Cassandra does not depend on HDFS, YARN, or ZooKeeper at all — it is a fully independent NoSQL database, more directly comparable to HBase (both are wide-column NoSQL stores) than to anything HDFS-dependent.

Cassandra — Peer-to-Peer Ring (No Master, By Design)



Every node is structurally EQUAL — no HMaster, no NameNode.
Nodes gossip cluster state to a few random peers every second;
that state spreads across the whole ring within a few rounds.

Cassandra — Peer-to-Peer Ring (No Master, By Design)

Compare this shape directly against every other diagram in this module — HDFS, MapReduce, YARN, and HBase all draw a clear master-at-the-top, workers-below hierarchy. Cassandra's ring has no top. This single structural difference is the real reason it doesn't belong on a "Hadoop ecosystem components" list: it isn't just a different PRODUCT, it's built on a fundamentally different COORDINATION PHILOSOPHY — decentralized gossip instead of centralized metadata — which is also exactly why Cassandra can keep accepting writes even while several nodes are simultaneously unreachable, a resilience property a single-master system has to work much harder to achieve.

> ⚠ **Interview-safe distinction**
>
> If asked to name "components of the Hadoop ecosystem," Cassandra should NOT be on that list — it's an independently-designed, masterless NoSQL database that happens to solve a similar problem to HBase (fast random access at scale) using a fundamentally different, decentralized architecture. Confusing the two is a common and easily avoided mistake.

> 🤔 **Check yourself — 2.4, 2.5, 2.6 & 2.7**
>
> Q1. What specific role does the Hive Metastore play, and what would happen if it became unavailable?
>
> Q2. Explain the difference between Hive's partitioning and bucketing in your own words.
>
> Q3. Why is HBase a better fit than plain HDFS for looking up one specific user's record instantly?
>
> Q4. Why is it architecturally inaccurate to call Cassandra part of the "Hadoop ecosystem"?
>
> **Hands-on Challenge:**
>
> Draw (on paper or in a diagram tool) the full classic Hadoop stack as a stack of layers: HDFS at the bottom, YARN above it, MapReduce/Hive/Pig above that, with HBase drawn as a separate tower standing NEXT TO the stack (also resting on HDFS) rather than stacked inside it. Label which tools depend on ZooKeeper.

MODULE 3 — The Spark Ecosystem, Component by Component

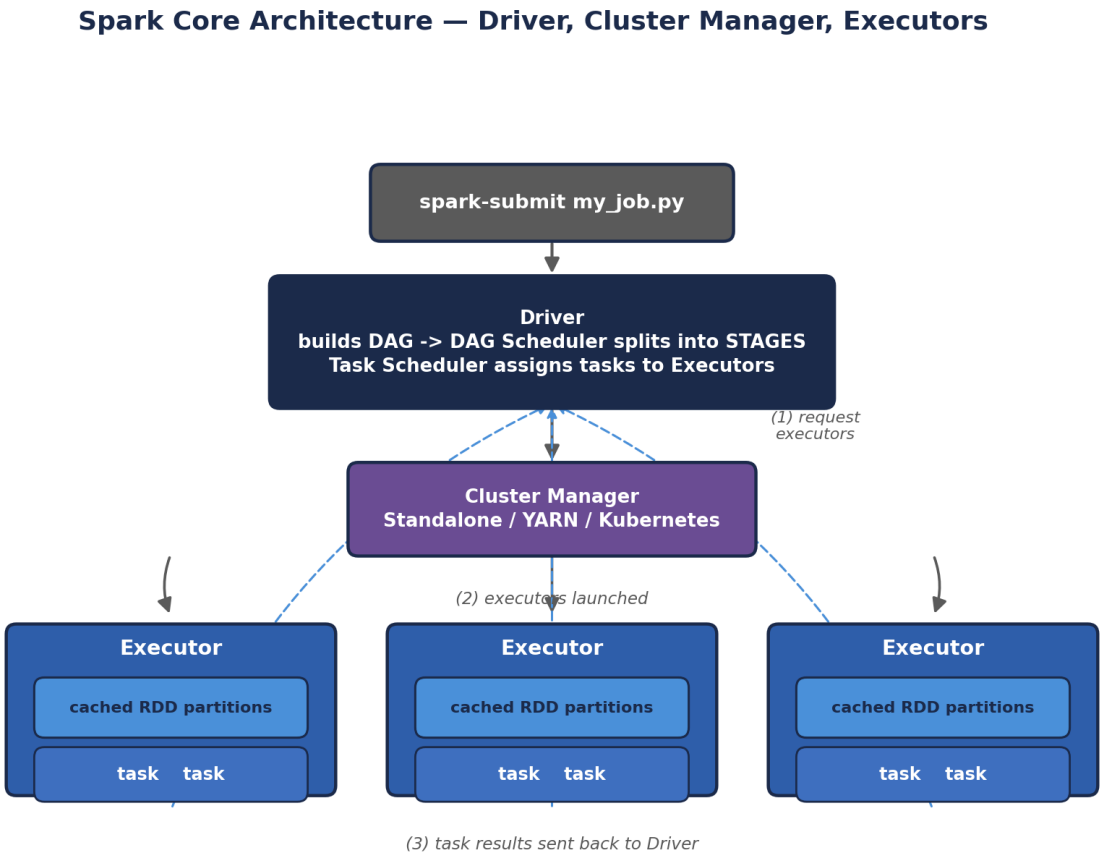
This is the engine INT315 spends the whole semester on. Where Module 2 was a refresher, this module is your first real architectural tour of Spark — worth reading closely even if some pieces (RDDs, Spark SQL) will be covered again in later units, because seeing the WHOLE ecosystem laid out at once, next to its Hadoop counterparts, makes each individual unit easier to place.

3.1 Spark Core — The Execution Engine

Spark Core is the foundational engine every other Spark component (SQL, Streaming, MLlib, GraphX) is built on top of. Its central architectural idea is the RDD (Resilient Distributed Dataset) — an immutable, partitioned collection of data spread across the cluster, along with the LINEAGE (the sequence of transformations) that produced it.

Architecture

- Driver — the process running your main Spark program (written in Scala, Python, Java, or R). It builds the logical execution plan as a DAG (Directed Acyclic Graph) of transformations, splits that DAG into stages and tasks, and coordinates their execution — this is the Spark-specific ApplicationMaster role referenced in Module 2.3.
- Cluster Manager — negotiates resources on the Driver's behalf; can be Spark's own Standalone manager, YARN, or Kubernetes (Module 3.6 covers the trade-offs).
- Executors — worker processes launched on cluster nodes for the lifetime of an application; each runs the actual tasks assigned to it and holds any RDD partitions cached in its memory.
- DAG Scheduler — inside the Driver; translates the logical sequence of RDD transformations into a physical plan of STAGES, splitting the DAG at shuffle boundaries (points where data must move across the network between partitions).
- Task Scheduler — inside the Driver; takes each stage's tasks and assigns them to specific Executors, handling retries if a task fails.



Spark Core Architecture — Driver, Cluster Manager, Executors

A detail worth calling out that the diagram can only hint at: Executors are launched ONCE per application and stay alive for its entire duration, running many tasks over time and — crucially — holding cached RDD partitions in memory BETWEEN stages. This is a structural difference from classic MapReduce, where each task historically ran in its own freshly-launched JVM process with no memory carried over to the next stage. Reusing long-lived Executor processes avoids repeated JVM startup overhead and is precisely what makes keeping data "in memory across stages" (Module 1's headline claim) physically possible in the first place.

Transformations vs. Actions — Spark's lazy evaluation model

Spark deliberately does NOT execute anything the moment you write a line of code. Operations split into two categories:

- Transformations (e.g., map, filter, groupByKey) — lazily build up the DAG of what SHOULD happen, without actually touching data yet.
- Actions (e.g., collect, count, saveAsTextFile) — trigger actual execution: only when an action is called does Spark walk the accumulated DAG backwards and execute the necessary transformations.

```
# PySpark: lazy transformations, then a triggering action
rdd = sc.textFile("hdfs:///user/student/data/access.log")
```

```
errors = rdd.filter(lambda line: "ERROR" in line)      # transformation --
nothing runs yet
error_words = errors.flatMap(lambda line: line.split())  # transformation --
still nothing runs

count = error_words.count()   # ACTION -- this is the moment Spark actually
executes the DAG
print(count)
```

The Chef Who Won't Start Cooking Until You Actually Order

An RDD transformation is like handing a chef a recipe card -- "first dice the onions, then saute them, then add the sauce." The chef reads it, nods, sets it on the counter, and does absolutely nothing else. You can hand over ten more recipe cards, each building on the last, and the chef just keeps stacking them.

Only when you actually say "I'll have that, please" (an ACTION like .collect() or .count()) does the chef finally look at the WHOLE stack of recipe cards at once, plan the most efficient way to execute them all in sequence, and start cooking. This laziness isn't procrastination -- it lets Spark see the entire pipeline before committing to a plan, so it can skip unnecessary work and optimize the whole DAG at once, rather than blindly executing each instruction the moment it's written.

Why lineage-based fault tolerance is the headline feature

Because the Driver keeps the full lineage graph of how each RDD was derived, if an Executor holding a cached partition dies mid-job, Spark doesn't need a pre-existing replicated backup copy (HDFS's approach) — it simply re-runs the recorded chain of transformations for JUST that lost partition, recomputing it from source or from the nearest available cached ancestor. This is consistently the first architectural fact interviewers expect a Spark candidate to explain confidently.

Check yourself — 3.1

Q1. What's the difference between a transformation and an action, and why does that distinction matter for performance?

Q2. In Spark's architecture, which component plays the same role YARN's ApplicationMaster plays for a generic application?

Q3. Explain how Spark recovers from a lost Executor without needing pre-replicated copies of every RDD partition.

Hands-on Challenge:

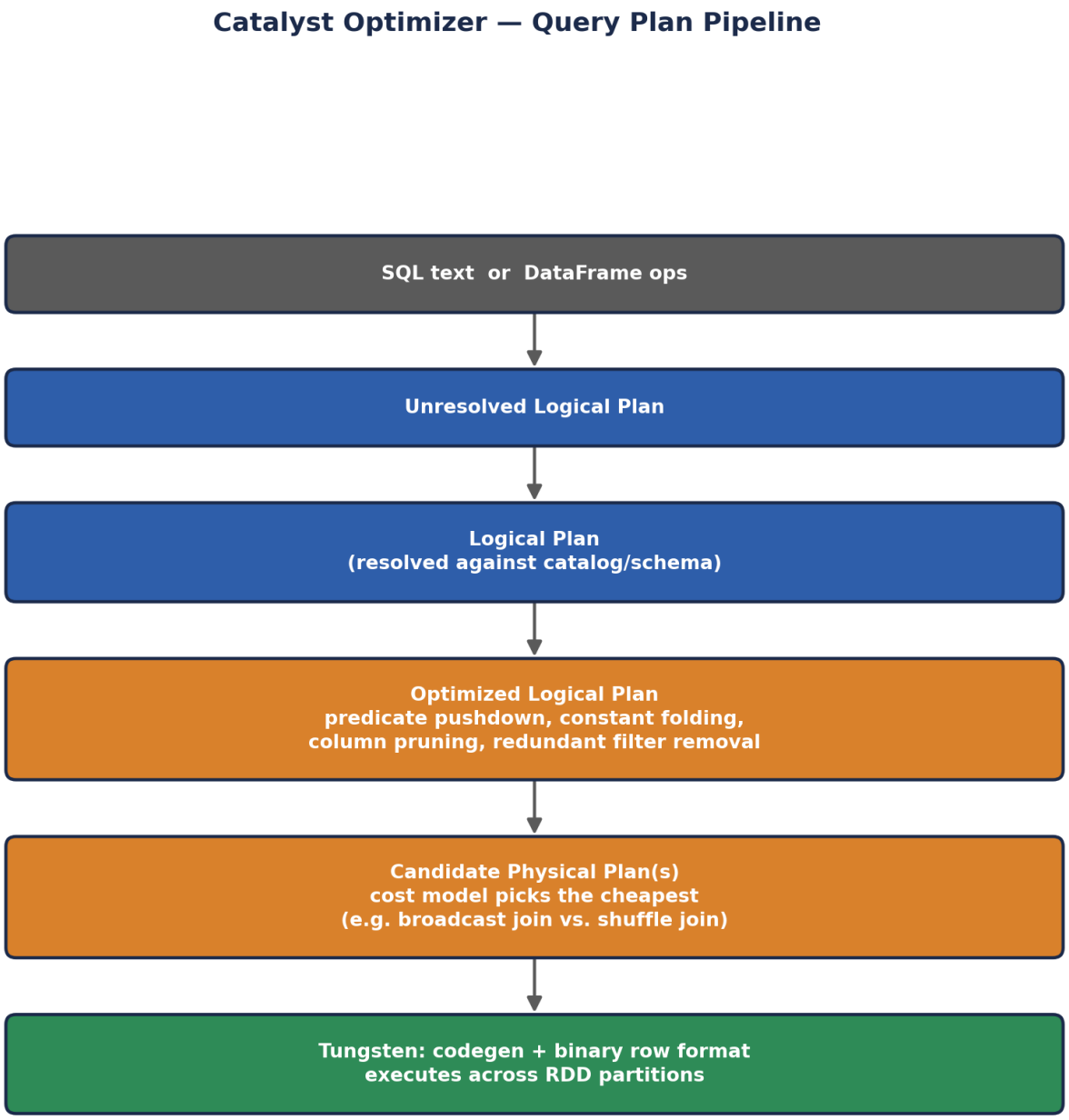
Write a short PySpark or Scala snippet with at least three chained transformations followed by one action, then call .toDebugString() (Scala RDD) or explain() (DataFrame) on it to see Spark's own internal representation of the lineage/plan you just built.

3.2 Spark SQL, DataFrames & the Catalyst Optimizer

Spark SQL is Spark's answer to Hive (Module 2.4) — a way to query structured data using familiar SQL, or Spark's DataFrame API (a higher-level, schema-aware abstraction over RDDs, conceptually similar to a pandas DataFrame but distributed across the cluster).

Architecture — how a query actually gets fast

- DataFrame / Dataset API — a structured, schema-aware layer on top of RDDs; unlike raw RDDs, Spark SQL KNOWS the column names and types, which unlocks the optimizations below.
- Catalyst Optimizer — Spark SQL's query planner; it takes your SQL query or DataFrame operations, builds a logical plan, applies rule-based optimizations (predicate pushdown, constant folding, redundant filter removal), then explores multiple physical execution strategies and picks the cheapest one — conceptually similar to how a traditional relational database's query optimizer works, but built specifically for distributed execution.
- Tungsten — Spark's physical execution engine layer, responsible for low-level performance: efficient binary memory layout of data (avoiding the overhead of JVM object headers per row), and generating optimized JVM bytecode at runtime for a given query plan rather than relying on generic, interpreted operators.



Catalyst Optimizer — Query Plan Pipeline

The step most students underestimate is "physical planning": Catalyst doesn't just pick ONE way to execute your query, it generates SEVERAL candidate physical plans (for example, a join could be executed as a broadcast join if one side is small enough to fit in memory, or as a shuffle join if both sides are large) and uses a cost model — informed by table statistics — to choose the cheapest one before a single task ever runs. This is conceptually identical to what a traditional RDBMS query optimizer (like PostgreSQL's or Oracle's) does, just re-engineered to reason about distributed, partitioned execution across many machines instead of a single machine's disk and memory.

```
# PySpark: DataFrame API and equivalent SQL, both go through the same Catalyst
optimizer
df = spark.read.parquet("hdfs:///warehouse/orders")

df.filter(df.order_date == '2026-08-01') \
  .groupBy('customer_id') \
  .sum('amount') \
  .show()

# The exact same logic, expressed as SQL against a registered temp view
df.createOrReplaceTempView('orders')
spark.sql("""
  SELECT customer_id, SUM(amount)
  FROM orders
  WHERE order_date = '2026-08-01'
  GROUP BY customer_id
""").show()

# See exactly what Catalyst decided to do
df.filter(df.order_date == '2026-08-01').explain(True)
```

The Architect Who Redraws the Blueprint Before Building

Hive's classic approach (Module 2.4) is like handing a construction crew your building request and having them start building floor by floor, roughly in the order you asked. Catalyst is a licensed architect who takes your blueprint request FIRST, redraws it for maximum efficiency -- moving load-bearing walls (predicate pushdown means filtering data as early and as close to the source as possible, rather than waiting until the end), removing rooms nobody asked for (redundant filter removal) -- and only THEN hands the optimized blueprint to the construction crew.

Tungsten is that same construction crew switching from hand-cutting every plank of wood on-site to using pre-fabricated, machine-optimized components -- doing the equivalent physical job, but with far less per-row overhead, because it's generating tightly optimized machine code specifically for YOUR blueprint rather than using slow, generic, one-size-fits-all tools.

Why this is the most common Spark-vs-Hive interview angle

"Why is Spark SQL usually faster than Hive-on-MapReduce, even for a similar-looking query?" tests exactly this layer: in-memory execution (Module 3.1's core advantage) COMBINED with a cost-based query optimizer (Catalyst) and low-level, code-generated execution (Tungsten) that Hive's original MapReduce-based execution simply never had. Modern Hive-on-Spark or Hive-on-Tez narrows this gap by borrowing similar ideas, which is itself a good example of how these ecosystems have converged over time.

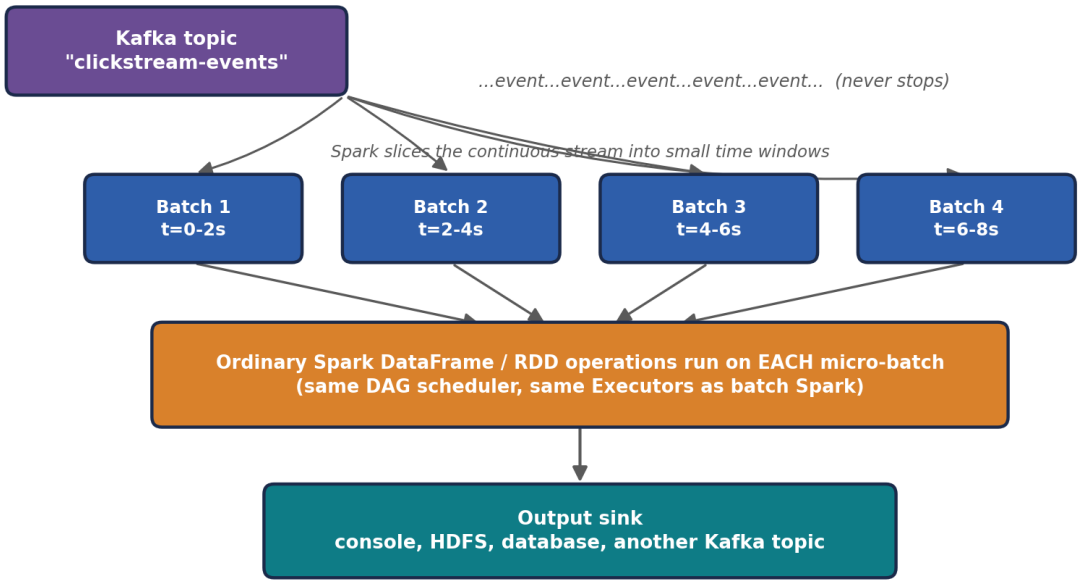
3.3 Spark Streaming & Structured Streaming — Real-Time Processing

Nothing in Module 2's classic Hadoop stack was designed for continuously arriving data — MapReduce and Hive both assume a finite, already-collected dataset. Spark Streaming (and its modern successor, Structured Streaming) is Spark's answer to real-time processing, and is most commonly paired with Apache Kafka as the data source (INT315 Unit V).

Two generations, one underlying idea

- Spark Streaming (DStreams, the original API) — processes data in micro-batches: it slices the incoming stream into small time-windowed RDDs (e.g., every 2 seconds) and runs ordinary Spark batch operations on each micro-batch in turn. Conceptually, this is "batch processing run very frequently" rather than true record-by-record streaming.
- Structured Streaming (the modern API) — expresses streaming computations using the SAME DataFrame/Dataset API as Spark SQL, treating an incoming stream as an unbounded, continuously-appending table. Spark itself figures out how to incrementally update the result as new data arrives, which unifies batch and streaming code into one programming model.

Micro-Batch Model — DStreams and Structured Streaming Alike



Micro-Batch Model — DStreams and Structured Streaming Alike

This diagram is the key to understanding WHY Spark Streaming was architecturally elegant even in its original DStreams form: rather than building a completely separate streaming engine from scratch, it reuses Spark Core's exact same DAG scheduler, task scheduler, and Executors (Module 3.1) — a micro-batch is, underneath everything, just an ordinary Spark job triggered automatically and repeatedly. Structured Streaming keeps this same micro-batch execution model under the hood by default, but hides the batching entirely behind the

DataFrame API, so your code reads as if it's simply querying one continuously growing table rather than manually managing a sequence of small jobs.

```
# Structured Streaming: reading from Kafka, using the SAME DataFrame API as
batch Spark SQL
stream_df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "clickstream-events")
    .load())

parsed = stream_df.selectExpr("CAST(value AS STRING) as json_str")

query = (parsed.writeStream
    .format("console")
    .outputMode("append")
    .start())

query.awaitTermination()
```

The Difference Between a Mail Truck and a Live News Ticker

Classic Spark Streaming's micro-batch model is like a mail truck that does its rounds every two minutes, picking up whatever letters have piled up since its last visit and processing that small batch together -- fast, but still fundamentally "collect a little pile, then process the pile," repeated very frequently.

Structured Streaming reframes the whole mental model: instead of thinking about trucks and piles at all, you write your logic as if you're simply querying an ever-growing table that new rows keep quietly appending to -- the same SELECT/GROUP BY you'd write against a static table -- and Spark handles the truck logistics invisibly underneath, incrementally updating your answer as new rows arrive.

Where this fits the industry landscape

Structured Streaming directly competes with (and is commonly paired alongside) Kafka Streams and Apache Flink in real production streaming architectures. Spark's specific advantage is that the exact same DataFrame code, skills, and even literal functions can be reused across a company's batch AND streaming pipelines, reducing the amount of separate streaming-specific expertise a data team needs to maintain.

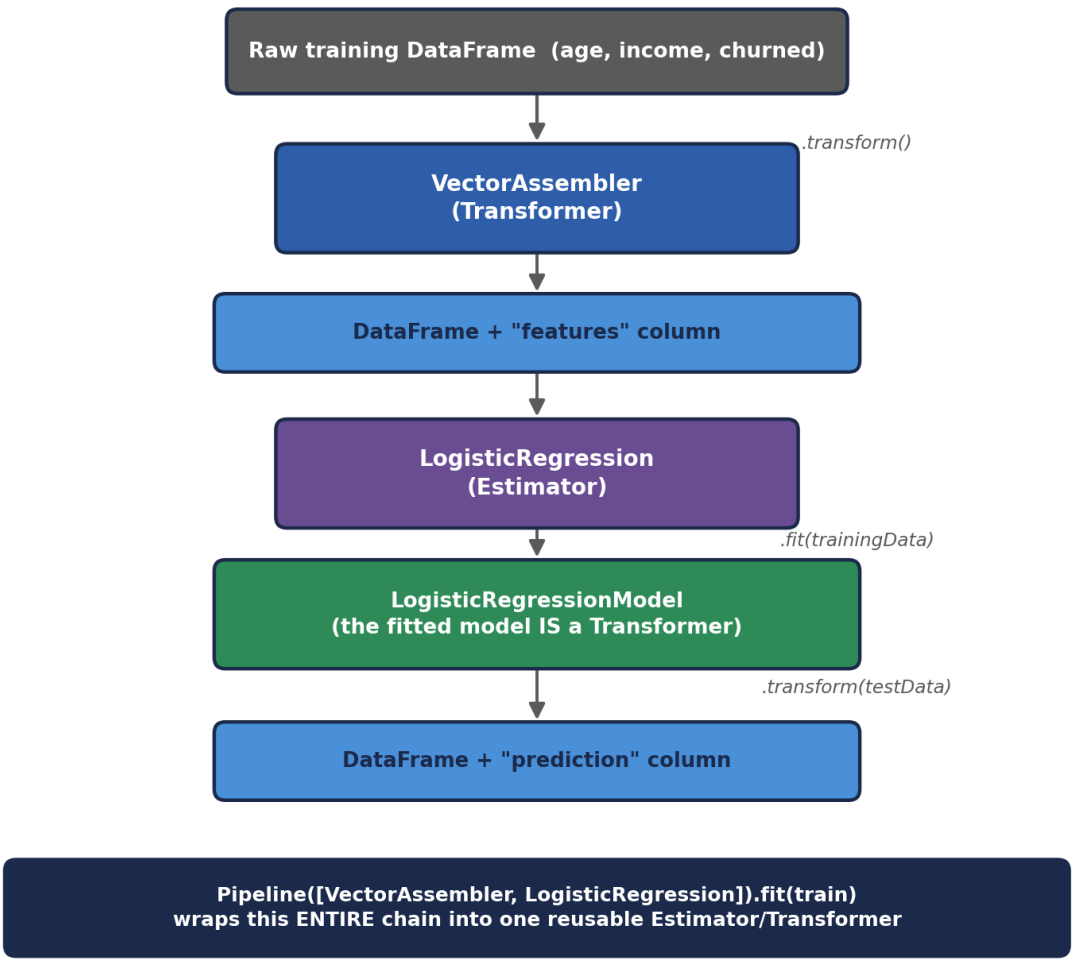
3.4 Spark MLlib — Machine Learning at Scale

MLlib is Spark's distributed machine learning library, built to train models on datasets far too large to fit on one machine's RAM — directly extending Module 3.1's core RDD/DataFrame execution model to iterative ML algorithms, which are exactly the kind of repeated-pass-over-the-same-data workload where Spark's in-memory advantage over MapReduce is most dramatic.

Architecture concept: the Pipeline API

MLlib organizes ML workflows using a small number of composable building blocks, modeled loosely on scikit-learn's API but designed for distributed DataFrames:

- Transformer — an algorithm that transforms one DataFrame into another (e.g., a trained model producing predictions, or a feature-encoding step).
- Estimator — an algorithm that FITS on a DataFrame to produce a Transformer (e.g., calling .fit() on a LogisticRegression estimator produces a trained LogisticRegressionModel, which is a Transformer).
- Pipeline — chains multiple Transformers and Estimators together into a single, reusable workflow — feature engineering steps followed by a model — that can be fit and applied as one unit.



MLlib Pipeline — Estimators, Transformers & fit()/transform()

The subtlety worth sitting with here: an Estimator's .fit() call is the only place actual DISTRIBUTED COMPUTATION happens across the training data — that's where Spark launches Executors to iterate over partitions and update model coefficients. Once fit() has produced a trained model (which is just a Transformer holding those learned coefficients), applying it to new data via .transform() is comparatively cheap — no further training, just running the already-learned formula across each row in parallel. This fit-once, transform-many-times pattern is exactly how the same trained model gets reused across every future batch of incoming data without retraining it each time.

```
# PySpark MLlib: a minimal pipeline -- feature assembly, then logistic
regression
```

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

assembler = VectorAssembler(inputCols=['age', 'income'], outputCol='features')
lr = LogisticRegression(featuresCol='features', labelCol='churned')

pipeline = Pipeline(stages=[assembler, lr])
model = pipeline.fit(training_df)      # Estimator.fit() -> Transformer
predictions = model.transform(test_df) # Transformer.transform() -> new
DataFrame

predictions.select('customer_id', 'prediction').show()
```

This is directly relevant to INT315 Unit VI: algorithms like Linear Regression, Logistic Regression, Decision Trees, K-means, SVM, and Naive Bayes are all implemented as Estimators following exactly this pattern, and evaluated using metrics like a confusion matrix, R-squared, RMSE, and MAE — all computed in a distributed fashion across the cluster rather than on a single machine's memory.

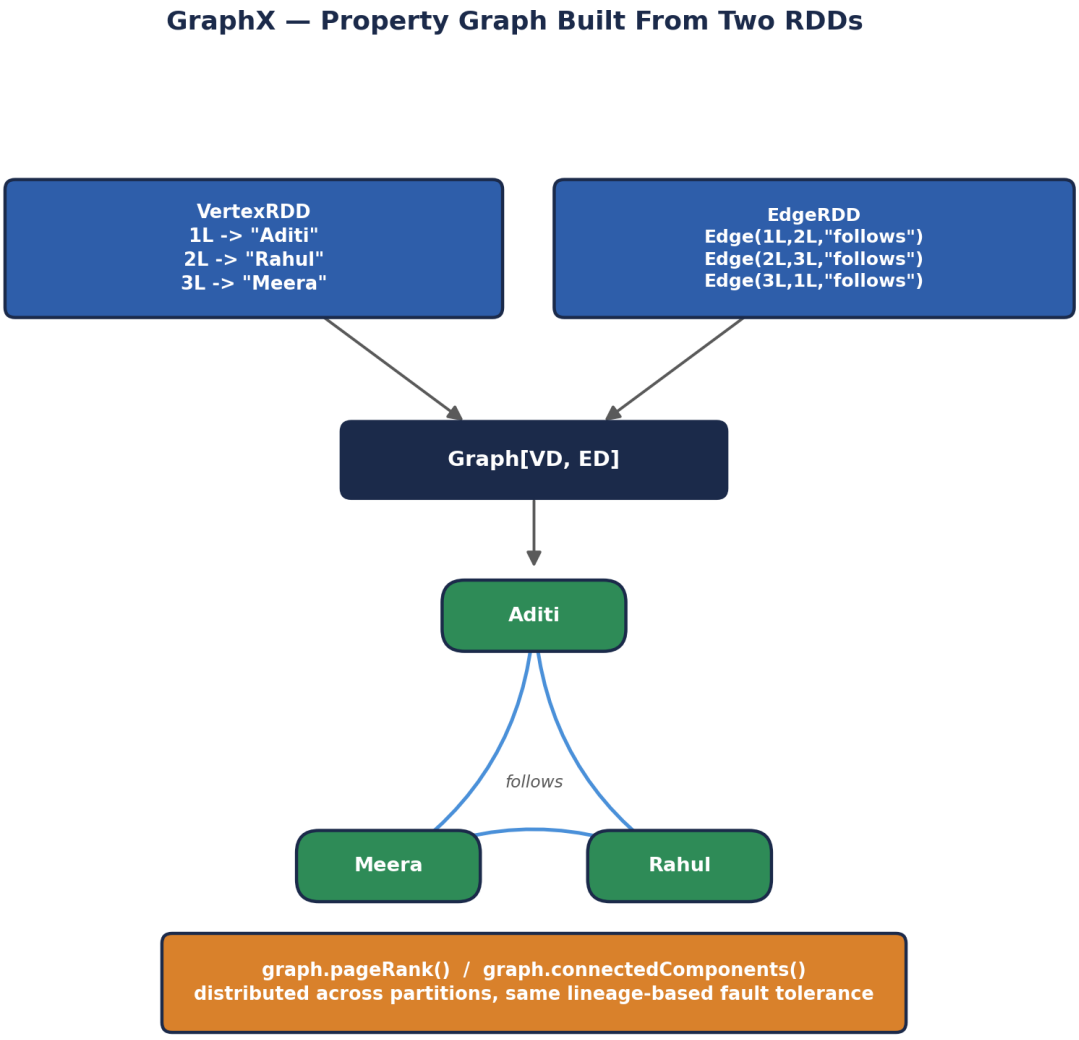
> **Why this matters more than it sounds**
>
> Classic MapReduce is a genuinely poor fit for iterative ML training, since most ML algorithms need MANY passes over the same data (gradient descent, for example, repeatedly recomputes gradients over the same training set). Each pass in MapReduce would mean a fresh round of costly disk writes and reads; Spark keeps the training data cached in memory across iterations, which is a major reason distributed ML at scale became practical.

3.5 Spark GraphX — Graph-Parallel Processing

GraphX extends Spark's RDD abstraction to graphs — data structured as vertices (nodes) and edges (relationships between nodes), useful for problems like social network analysis, recommendation systems, and web-link analysis (PageRank-style algorithms). This is genuinely NEW territory relative to INT312 — nothing in the classic Hadoop ecosystem module above has a direct equivalent.

Core architecture idea: the Property Graph

GraphX represents a graph as a Property Graph — vertices and edges that can each carry arbitrary properties (attributes) — internally built from two specialized RDDs (a VertexRDD and an EdgeRDD), letting graph-parallel operations reuse Spark Core's same distributed, fault-tolerant execution machinery rather than requiring an entirely separate processing system.



GraphX — Property Graph Built From Two RDDs

Notice both RDDs in this diagram are ordinary Spark RDDs — GraphX doesn't invent a new distributed data structure from scratch, it composes two already-battle-tested Module 3.1 building blocks (partitioned, fault-tolerant collections with lineage-based recovery) into a graph-shaped abstraction. This is a recurring architectural pattern across the whole Spark ecosystem: Spark SQL's DataFrames, MLlib's distributed vectors, and GraphX's property graphs are all, underneath their specialized APIs, built from the exact same RDD foundation from Module 3.1 — one execution engine, several purpose-built abstractions layered on top.

```
# Scala: constructing a small property graph and running a built-in algorithm
import org.apache.spark.graphx._

val vertices = sc.parallelize(Seq(
  (1L, "Aditi"), (2L, "Rahul"), (3L, "Meera")))
val edges = sc.parallelize(Seq(
```

```
  Edge(1L, 2L, "follows"), Edge(2L, 3L, "follows"), Edge(3L, 1L, "follows")))

val graph = Graph(vertices, edges)

// Run PageRank -- a graph-parallel algorithm built into GraphX
val ranks = graph.pageRank(0.0001).vertices
ranks.collect().foreach(println)
```

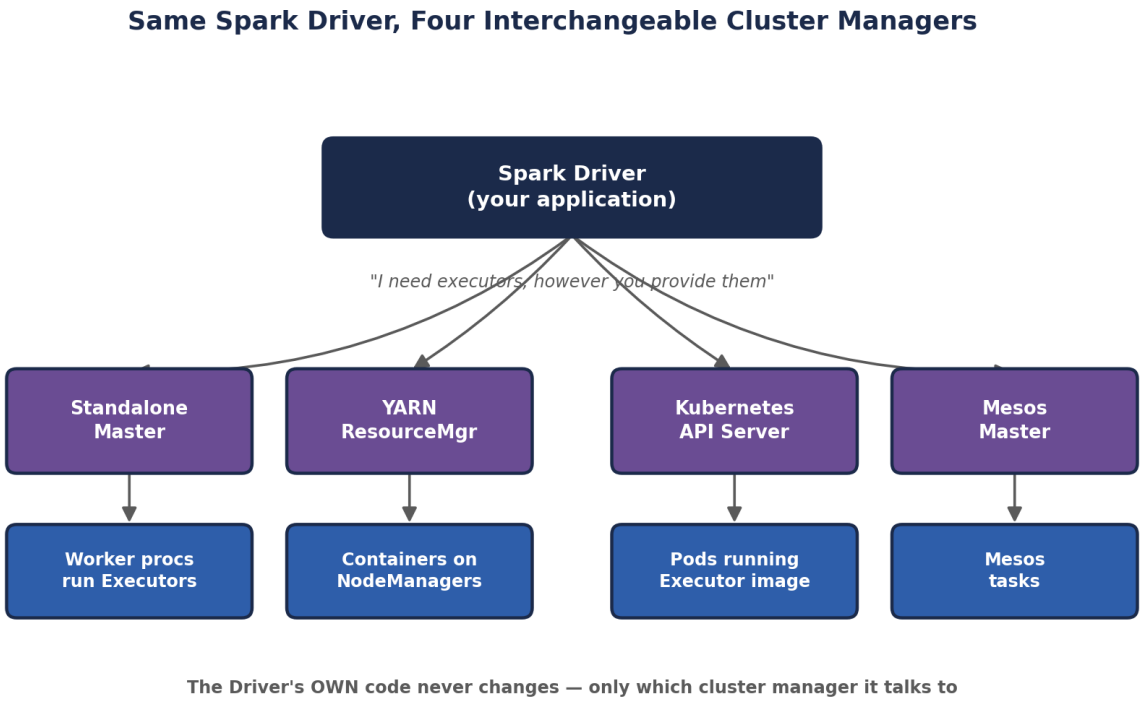
From Spreadsheets to Social Maps

Everything else in Spark so far -- DataFrames, MLlib -- thinks in terms of ROWS, largely independent of one another. GraphX asks a fundamentally different kind of question: not "what does this row say" but "who is CONNECTED to whom, and how does influence or information flow across those connections?" It's the difference between analyzing a spreadsheet of people and analyzing an actual map of who follows, trusts, or influences whom -- a shape of problem the classic Hadoop ecosystem never had a dedicated tool for at all.

3.6 Spark's Cluster Manager Options

Recall from Module 2.3 that Spark doesn't have to bring its own resource manager — it can run under several different ones, a flexibility classic MapReduce never had (MapReduce is tightly bound to Hadoop's YARN).

- Standalone — Spark's own simple, built-in cluster manager (the Master/Worker setup from the Prerequisites handbook). Easiest to set up for learning and small dedicated clusters, but lacks the multi-framework, multi-tenant resource negotiation YARN or Kubernetes provide.
- YARN — lets Spark applications share a Hadoop cluster's resources alongside MapReduce, Hive, and other YARN-aware frameworks; the most common choice for organizations with an existing Hadoop investment, since it reuses Module 2.3's ResourceManager/NodeManager/ApplicationMaster architecture directly, with the Spark Driver acting as the ApplicationMaster.
- Kubernetes — increasingly the default choice for NEW deployments, especially cloud-native ones; runs each Spark Executor as a Kubernetes pod, letting Spark share infrastructure and tooling with the broader container ecosystem most modern engineering organizations already use for everything else.
- Mesos — an early, general-purpose cluster resource manager (not Hadoop-specific) that saw significant adoption before Kubernetes became dominant; still found in some longer-established deployments but rarely chosen for new ones today.



Same Spark Driver, Four Interchangeable Cluster Managers

This diagram is the architectural payoff of Module 3.1's design: because the Driver only ever talks to an abstract "Cluster Manager" interface — asking for executors and receiving them, regardless of how they were physically provisioned — the exact same Spark application code runs unmodified whether it's submitted to a Standalone cluster in a classroom lab, a YARN cluster at an enterprise with an existing Hadoop investment, or a Kubernetes cluster on a public cloud. This portability is a genuinely practical advantage over classic MapReduce, which has no equivalent notion of a swappable cluster manager at all.

> **What real companies actually pick, and why**
>
> Organizations already running a mature Hadoop cluster commonly run Spark ON YARN, minimizing operational overhead by reusing existing infrastructure and access controls. Cloud-native companies building fresh, especially on managed services like Databricks, Amazon EMR, or Google Dataproc, increasingly default to Kubernetes-based or fully managed cluster provisioning instead, treating the underlying resource manager as an implementation detail the platform handles for them.

3.7 PySpark & Language Bridges — How Python Talks to a JVM Engine

Spark itself is written in Scala and runs on the JVM (tying directly back to the Prerequisites handbook's JVM memory discussion). Yet PySpark — writing Spark applications in Python — is enormously popular, because most of the data science and ML world lives in Python. Understanding HOW that bridge works avoids a lot of confusion later.

When you run PySpark, TWO processes are actually involved: a Python process running your script, and a JVM process running the real Spark engine. For DataFrame and SQL operations, your Python code mostly just builds up a logical query plan and hands it to the JVM side, where Catalyst and Tungsten (Module 3.2) do the actual heavy lifting — so DataFrame-based PySpark code runs at close to native Scala Spark speed. For custom Python functions applied row-by-row (UDFs) or raw RDD operations, data has to be SERIALIZED (recall the Prerequisites handbook's serialization section) and shipped between the JVM and a separate Python worker process via a bridge historically called Py4J, which introduces real, measurable overhead compared to native JVM code.

The practical takeaway for INT315 Unit VI

Prefer Spark's built-in DataFrame operations and MLlib's Estimators over hand-written Python UDFs whenever possible — the former stay entirely on the fast, optimized JVM side; the latter pay a real cross-process serialization tax every time they run. This single fact explains a large share of real-world PySpark performance-tuning advice.

Check yourself — 3.4, 3.5, 3.6 & 3.7

Q1. In MLlib's Pipeline API, what's the difference between an Estimator and a Transformer?

Q2. Why was iterative MapReduce-based machine learning historically impractical at scale, and what does Spark do differently?

Q3. Name one problem GraphX is suited for that has no natural equivalent among the classic Hadoop ecosystem tools in Module 2.

Q4. Why does a PySpark DataFrame operation typically run much faster than an equivalent PySpark UDF applied row-by-row?

Hands-on Challenge:

Pick any small dataset idea (a handful of users and their follow relationships works well) and sketch it as a property graph on paper: list the vertices with one property each, and the edges with a label, exactly as you would define them for GraphX's Graph() constructor.

MODULE 4 — Head-to-Head: Component Mapping & Industry Standards

With both ecosystems laid out architecturally, this module puts them side by side directly — first mapping components onto their closest counterpart, then comparing the two ecosystems as a whole against the criteria that actually drive real architecture decisions in industry.

4.1 Direct Component Mapping

Aspect	Hadoop Ecosystem Component	Closest Spark Ecosystem Counterpart
Storage	HDFS (also HBase/Cassandra for random access)	No native storage layer — reads/writes HDFS, S3, or any compatible store
Batch processing engine	MapReduce (JobTracker/TaskTracker or YARN-based)	Spark Core (RDDs / DataFrames, in-memory DAG execution)
SQL access	Hive (HiveQL, Metastore, MR/Tez execution)	Spark SQL (DataFrame API + SQL, Catalyst + Tungsten)
Resource management	YARN (ResourceManager / NodeManager)	Can reuse YARN, or use Kubernetes / Standalone
Real-time / streaming	No native equivalent (batch-only by design)	Spark Streaming / Structured Streaming
Machine learning at scale	Mahout (largely superseded, rarely used today)	Spark MLlib
Graph processing	No native equivalent	Spark GraphX
Scripting / dataflow	Pig (Pig Latin)	Spark DataFrame/RDD API (Python, Scala, SQL)
Coordination service	ZooKeeper	ZooKeeper (still used by some Spark deployments, e.g. HA setups)
Workflow scheduling	Oozie	Apache Airflow (industry-common, ecosystem-external to Spark itself)

⚠ **The mapping that trips people up**

Notice Spark has NO row under "Storage" of its own — this is the single most important nuance in the whole comparison. Spark is a COMPUTE engine, full stop; it always reads from and writes to some external storage system (HDFS, S3, HBase, a relational database, Kafka for streams). Saying "Spark

Aspect	Hadoop Ecosystem Component	Closest Spark Ecosystem Counterpart

> replaced HDFS" is factually wrong and an immediate red flag in a technical interview — the accurate statement is that Spark replaced MapReduce specifically, as Module 1 established.

4.2 Performance & Fault Tolerance

Aspect	Hadoop MapReduce	Spark
Intermediate data	Written to disk between every stage	Kept in memory (RDDs) across stages whenever possible
Fault tolerance mechanism	Replicated data on disk (HDFS's 3x replication)	Lineage-based recomputation of lost partitions
Typical iterative-job speed	Baseline — slow on multi-pass workloads	Commonly 10-100x faster on iterative/multi-stage workloads
Startup latency per job	Relatively high (JVM startup, disk-based staging)	Lower, especially for long-lived applications reusing Executors
Best-case workload	Simple, single-pass, very large batch scans	Iterative ML, interactive queries, streaming, multi-stage pipelines

4.3 Cost, Ease of Use & Ecosystem Maturity

Aspect	Hadoop Ecosystem	Spark Ecosystem
Language support	Java-centric historically; Hive/Pig add SQL-like access	First-class Scala, Python, Java, R, and SQL support
Learning curve	Steeper for hand-written MapReduce; gentler via Hive/Pig	DataFrame API feels familiar to SQL/pandas users quickly
Memory/hardware cost	Lower RAM requirements; leans on disk more	Higher RAM requirements to fully exploit in-memory advantage
Ecosystem maturity	Very mature, battle-tested since the mid-2000s	Mature and dominant since the mid-2010s onward
Community / hiring market	Still present, especially at legacy enterprises	Larger, faster-growing demand in current job postings

4.4 A Decision Framework — Which One For Which Job?

In practice, the honest industry answer is rarely "pick one" — most real deployments run BOTH, layered together. A grounded, defensible framework:

- Use HDFS (or a cloud object store) for storage almost regardless of which compute engine sits on top — this decision is largely independent of the batch-vs-Spark question.
- Use Spark instead of hand-written MapReduce for essentially all new batch processing development — the performance and productivity gap is large enough that greenfield MapReduce code is now rare in industry.
- Use Hive (or Spark SQL against the same Hive Metastore) when analysts need SQL access to a large, established data warehouse with well-defined, governed schemas.
- Use HBase or Cassandra, not Spark, when the requirement is millisecond-latency random reads/writes on individual records — Spark's batch/micro-batch DataFrame model is the wrong tool for that specific job, even though Spark can READ from HBase/Cassandra as a data source.
- Use Structured Streaming (with Kafka) specifically when the business requirement has a genuine real-time latency constraint — fraud detection, live dashboards, alerting — rather than defaulting to streaming for its own sake when a simple nightly batch job would satisfy the actual requirement.

The single sentence to remember for interviews

"Hadoop and Spark aren't rivals fighting over the same job — Hadoop's HDFS is commonly the storage foundation Spark computes on top of, YARN is commonly the resource manager Spark runs under, and Spark is commonly the faster, more general-purpose engine that replaced hand-written MapReduce specifically for the processing layer." This one sentence, said confidently, outperforms a much longer answer that just lists features.

MODULE 5 — The Grand Unified Analogy: Two Postal Systems

Every individual comic box in this handbook has used its own small metaphor. This module pulls all of it together into ONE extended story, so the whole architecture — both ecosystems, side by side — sits in your head as a single coherent picture rather than a pile of separate facts.

Two Postal Systems Serving the Same City

Imagine a sprawling city called Big Data City, and two competing postal companies that both need to store, sort, and deliver an enormous, ever-growing volume of mail across it.

THE HADOOP POSTAL COMPANY built the city's original infrastructure. Its WAREHOUSE NETWORK (HDFS) is a system of branch warehouses spread across the city, each holding physical copies of mail, with a central card-catalog desk (the NameNode) that never touches a single envelope itself, only ever telling you which warehouse has what. Its original SORTING CREW (MapReduce, run by JobTracker/TaskTracker) is famously careful: after every single sorting step, the crew insists on filing a fresh paper copy of their progress at the nearest warehouse, JUST IN CASE something goes wrong — reliable, but slow, because paperwork gets filed and re-fetched constantly. Eventually the company splits crew-scheduling duties (YARN's ResourceManager/NodeManager/ApplicationMaster) away from the sorting work itself, so multiple crews can share the same warehouses without stepping on each other. Around this core, a SQL-speaking front desk (Hive) lets ordinary customers request mail using plain English instead of sorting-crew jargon, and a SPEED COUNTER (HBase) opens up for customers who need a single specific letter fetched in seconds, not a whole warehouse search.

THE SPARK POSTAL COMPANY arrives later, doesn't build its own warehouses at all, and instead strikes a deal to use the SAME warehouse network already sitting across the city (Spark reads and writes HDFS, S3, and other existing storage). What it brings instead is a radically redesigned SORTING CREW (Spark Core) that keeps mail on rolling carts IN HAND between sorting steps rather than constantly re-filing paperwork at the warehouse, only filing something away when the carts are genuinely full — and if a cart gets lost, the crew doesn't need a backup copy sitting in a warehouse, because they kept a written recipe (lineage) of exactly how that cart's contents were sorted, and can just re-sort it from scratch in seconds. This crew also opens its own SQL-speaking front desk (Spark SQL) with a much sharper dispatcher (Catalyst) who redraws the most efficient sorting route before a single envelope moves, a LIVE INTAKE COUNTER (Structured Streaming) for mail that never stops arriving, a STATISTICS DEPARTMENT (MLlib) that can spot patterns across millions of letters at once, and — something the original company never built at all — a RELATIONSHIP-MAPPING DESK (GraphX) that traces who's been sending mail to whom across the entire city.

Both companies, in the end, are still delivering mail across the SAME city, often even sharing the SAME warehouses and the SAME crew-scheduling office (YARN). The real difference the whole city eventually noticed wasn't a rivalry — it was that the second company's redesigned sorting crew got the SAME jobs done dramatically faster, and could additionally handle two entirely new kinds of requests (live streaming mail, and relationship mapping) the first crew was never built to take on at all.

How to use this story going forward

Whenever a new Spark concept shows up later in INT315 and you're not sure where it fits, ask: "is this part of the warehouse (storage), the sorting crew (compute engine), the crew-scheduling office (resource manager), or one of the specialized desks (SQL/streaming/ML/graph)?" That single question, traceable straight back to this one analogy, will correctly place almost everything you encounter for the rest of the course.

MODULE 6 — Unit-by-Unit: How INT312 Maps Onto INT315

Module 0 gave you the high-level bridge between the two courses. Now that both ecosystems have been laid out in full architectural detail, here's the precise, unit-by-unit mapping — useful both as a study aid and as a way to see exactly how much of INT315 is already familiar territory wearing a faster engine.

Aspect	INT312 (Big Data Fundamentals)	INT315 (Cluster Computing)
Unit I — Intro to Big Data & Hadoop	Unit I — Spark introduced directly against this same backdrop: limitations of the MapReduce you just learned motivate Spark's in-memory design	
Unit II — Hadoop Architecture, HDFS, NameNode/DataNode, JobTracker/TaskTracker	Unit I-II — Same HDFS storage concepts reused as-is; JobTracker/TaskTracker's YARN successor becomes one of Spark's cluster manager options	
Unit III — MapReduce Java code (Mapper/Reducer/Driver), YARN model	Unit II-III — Scala replaces hand-written Java Mapper/Reducer boilerplate; RDD transformations/actions replace the Map/Reduce split directly	
Unit IV — Apache Hive (installation, data types, bucketing, partitioning, HiveQL)	Unit IV — Spark SQL covers the same querying need with DataFrames + SQL, reusing partitioning/bucketing concepts, running faster via Catalyst/Tungsten	
Unit V — Apache HBase (architecture, data model, Java API, filtering, TTL)	No direct unit equivalent — HBase remains a valid data SOURCE Spark can read from; INT315 does not re-teach HBase itself	
Unit VI — Apache Cassandra (peer-to-peer architecture, CQL, replication)	No direct unit equivalent — same relationship as HBase: a possible external data source, not re-taught	
(no equivalent — new in INT315)	Unit V — Spark Streaming with Apache Kafka: genuinely new real-time processing territory	
(no equivalent — new in INT315)	Unit VI — Spark MLlib / PySpark ML: genuinely new distributed machine learning territory	

Two things stand out from this mapping. First, roughly two-thirds of INT315's early units are a faster re-implementation of concepts INT312 already introduced — HDFS storage, the Map/Reduce transformation idea, and SQL-style querying via partitioning and bucketing all carry over almost unchanged conceptually, just executed by a different, faster engine. Second, INT315's back half (Units V-VI) is where the genuinely new material lives:

real-time streaming and distributed machine learning have no INT312 counterpart at all, because classic Hadoop's ecosystem, as Module 2.6/3.3 established, was never designed to do either well.

Same Backpack, New Gear Added

Think of INT312 as packing your backpack with the essential gear for a long trek: a map of the terrain (Big Data concepts), a sturdy tent (HDFS), a reliable if slow stove (MapReduce), and a couple of specialized tools for specific terrain (Hive for SQL-shaped ground, HBase and Cassandra for fast-access rocky patches). INT315 doesn't ask you to repack the whole bag -- it swaps the slow stove for a much faster one (Spark) that still cooks on the same campsite (HDFS), and then hands you two entirely new tools you never needed on the last trip: a weather radio for constant live updates (Kafka streaming) and a set of binoculars for spotting patterns across the whole valley at once (MLlib).

Check yourself — Module 6

Q1. Which INT312 unit's core concept (HDFS or MapReduce) is reused MOST directly and unchanged in INT315, versus which is most dramatically redesigned?

Q2. Why do INT315's Units V and VI have no direct INT312 counterpart?

Q3. If an interviewer asked you to describe your own Big Data coursework in two sentences, how would you now summarize the INT312-to-INT315 progression?

Hands-on Challenge:

Write your own one-paragraph 'academic transcript summary' — the kind you might paste into a resume's coursework section — that accurately captures both courses together as a coherent, non-redundant skill progression rather than two disconnected classes.

Closing the Loop

You now have the full picture this handbook set out to build: the Hadoop ecosystem's storage, batch processing, resource management, and SQL/NoSQL layers, laid architecture-first; the Spark ecosystem's execution engine, SQL layer, streaming, ML, and graph capabilities, laid out the same way; a precise, industry-standard comparison between the two rather than a vague "Spark is better" claim; and an explicit map from what INT312 already gave you onto what INT315 is about to build on top of it.

Carry forward, above everything else, Module 1's precise framing: Spark did not replace Hadoop. It replaced MapReduce, the processing engine, while continuing to lean on HDFS for storage and YARN (among other options) for resource management. Every detailed architecture section in Modules 2 and 3 is really just that one sentence, unpacked component by component — and now, having unpacked it fully, you're genuinely ready for the rest of INT315 to build on solid ground.

Before Unit I of INT315 begins

Make sure you could, without notes, sketch the layered stack from Module 1 (storage / compute / resource manager / supporting tools), fill in the component mapping table from Module 4.1 from memory, and retell at least the shape of the Two Postal Systems analogy from Module 5 in your own words. If all three feel solid, you're not just ready for Unit I — you're ready to explain WHY it matters.